Министерство образования Республики Беларусь
Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Конструирование программного обеспечения»

Тема: «Разработка основной логики программы»

Выполнил:

Студент 2 курса

Группы ПО-17

Ковалев А.И.

Проверил:

Кулик А.Д.

Брест 2026

Цель работы

Написать основную логику программы: сделать игру, которая запускается при выборе пункта «Начать игру», и оформить новые данные в виде классов.

Условия выполнения

Код программы писал в онлайн-компиляторе, локально среду разработки не ставил. Итоговый код лежит в файле src/main.cpp, тесты в файле tests/test_main.cpp, а скрипт tests/run_tests.sh собирает тесты через g++ и запускает их.

Задание

По варианту 3 игра представляет собой симулятор ежедневной заботы о животных. Нужно реализовать классы Player, Animal и Food. Игрок каждый день делает определённое количество действий: покупает новых животных, ухаживает за теми, что уже есть, кормит их и закупает еду. Разные виды животных должны питаться разной едой. Все настройки из предыдущих лабораторных работ должны учитываться в игре.

Описание программы

За основу взята программа из второй лабораторной работы. Главное меню, настройки и работа с файлом settings.txt остались прежними, версия в пункте «О программе» теперь 3.1. Изменился пункт «Начать игру»: раньше он только выводил сообщение, а теперь запускает игру. Игра идёт 7 дней. В каждом дне у игрока есть ограниченное число действий, а в конце дня животные приносят доход, становятся голоднее и теряют настроение.

Первый класс, Food, описывает корм. У него есть название и цена одной порции:

```
class Food
{
public:
    Food(const string& name, int price) : name(name), price(price)
{}
    string getName() const { return name; }
    int getPrice() const { return price; }
private:
    string name;
    int price;
};
```

Поля name и price находятся в разделе private, поэтому снаружи их можно только прочитать через методы getName() и getPrice(). Конструктор принимает название и цену и записывает их в поля. Всего в игре пять видов корма: сено, зерно, овощи, трава и тофу, они собраны в списке, который возвращает функция foodList().

Чтобы разные животные ели разное, написана функция foodNameForSpecies(). У известных видов корм задан таблицей: корова ест сено, курица зерно, свинья овощи, овца траву, кицунэ тофу. Но в настройках можно добавлять любые новые виды, для них корм подбирается по названию:

```
map<string, string>::const_iterator it = table.find(species);
if (it != table.end())
    return it->second;

unsigned int sum = 0;
for (unsigned char c : species)
    sum += c;
return foodList()[sum % foodList().size()].getName();
```

Сначала вид ищется в таблице. Если его там нет, складываются коды всех символов названия, и остаток от деления на количество кормов даёт номер корма. Так для одного и того же названия корм всегда одинаковый, а у разных видов он, как правило, отличается.

Второй класс, Animal, описывает животное. У него есть вид, имя (например, «Корова 1»), корм и две характеристики от 0 до 100: голод и настроение.

```
Animal(const string& species, const string& name)
    : species(species), name(name),
foodName(foodNameForSpecies(species)),
      hunger(START_HUNGER), mood(START_MOOD) {}

void feed()
{
    hunger -= FEED_AMOUNT;
    if (hunger < 0)
        hunger = 0;
}
```

Конструктор по виду сразу определяет, какой корм нужен животному, и ставит начальные значения: голод 20 и настроение 70. Метод feed() уменьшает голод на 50, но не ниже нуля. Метод care() работает похоже: он повышает настроение на 30, но не выше 100. Метод passDay() вызывается в конце дня: голод растёт на значение из правил сложности, а настроение падает на 10.

```
bool isAlive() const { return hunger < MAX_STAT; }

int getDailyIncome() const
{
    if (hunger < INCOME_MAX_HUNGER && mood >= INCOME_MIN_MOOD)
        return incomeForSpecies(species);
    return 0;
}
```

Животное считается живым, пока голод меньше 100. Доход оно приносит только если сытое (голод меньше 50) и в хорошем настроении (не меньше 50). Размер дохода зависит от вида: например, корова приносит 12 в день, а курица 5. Поэтому игроку нужно не только кормить животных, но и заботиться о них.

Третий класс, Player, это игрок. В нём хранятся деньги, количество оставшихся действий, список животных, запас корма по видам и счётчики для имён животных. Каждое действие игрока это метод, который проверяет возможность и возвращает true или false. Например, кормление:

```
bool feedAnimal(int index)
{
    if (actionsLeft <= 0 || index < 0 || index >=
(int)herd.size())
        return false;
    string foodName = herd[index].getFoodName();
    if (getFood(foodName) <= 0)
        return false;
    foodStock[foodName]--;
    herd[index].feed();
    actionsLeft--;
    return true;
}
```

Метод сначала проверяет, что действия остались и номер животного верный, потом что на складе есть корм именно того вида, который ест это животное. Только после этого корм списывается, животное кормится и тратится одно действие. Если что-то не так, метод возвращает false и ничего не меняет, поэтому неудачная попытка не отнимает действие. Так же устроены методы careAnimal(), buyFood() и buyAnimal(). Корм продаётся упаковками по 5 порций, а цена нового животного зависит от вида.

В конце дня вызывается метод endDay():

```
for (const Animal& animal : herd)
    report.income += animal.getDailyIncome();
money += report.income;

vector<Animal> alive;
for (Animal animal : herd)
{
    animal.passDay(hungerPerDay);
    if (animal.isAlive())
        alive.push_back(animal);
    else
        report.diedNames.push_back(animal.getName());
}
herd = alive;
```

Сначала считается доход всех животных и прибавляется к деньгам. Потом каждое животное «проживает» день, и те, у кого голод дошёл до 100, не попадают в новый список alive, то есть погибают. Их имена записываются в отчёт за день, чтобы вывести их на экран.

Настройки из предыдущих лабораторных работ влияют на игру двумя способами. Первый: уровень сложности определяет правила, их возвращает функция getRules():

```
if (level == "Легко")
    rules = {250, 8, 15, 7};
else if (level == "Сложно")
    rules = {120, 5, 25, 7};
else
    rules = {180, 6, 20, 7};
```

Числа означают стартовые деньги, количество действий в день, прирост голода за день и количество дней. На лёгком уровне денег и действий больше, а животные голодают

медленнее. Второй способ: животные из настроек определяют стартовое стадо, а новые виды из настроек можно покупать в игре. Это видно в функции startGame():

```
GameRules rules = getRules(difficulty);
Player player(rules.startMoney, rules.actionsPerDay);

for (const auto& entry : animals)
    for (int i = 0; i < entry.second; i++)
        player.addAnimal(entry.first);
```

Для каждого вида из контейнера animals создаётся столько животных, сколько указано в настройках. Дальше запускается цикл по дням. В каждом дне функция playDay() показывает состояние фермы и меню действий (накормить, позаботиться, купить еду, купить животное, закончить день) и выбирает действие через switch с константами. День заканчивается, когда действия кончились или игрок выбрал «Закончить день». Игра идёт до конца 7-го дня или пока не погибнут все животные. В конце показываются деньги, количество животных и итоговый счёт, который равен деньгам плюс 20 очков за каждое животное.

Дополнительно в игру добавлено животное кицунэ. Это лиса из японских легенд: она ест тофу, стоит 80 монет и приносит 15 в день. Её особенность в том, что, как настоящая лиса, она каждый день съедает одну курицу. Чтобы кицунэ появилась на ферме, нужно добавить вид «Кицунэ» в настройках (пункт «Добавить новый вид животного») или купить её в игре, если этот вид уже есть в настройках. Кто кого съедает, решает метод kitsuneEats() класса Player:

```
for (const Animal& animal : herd)
{
    if (animal.getSpecies() == CHICKEN_SPECIES
        && (int)eaten.size() < kitsunes)
        eaten.push_back(animal.getName());
    else
        rest.push_back(animal);
}
```

Сначала метод считает, сколько кицунэ на ферме. Потом он просматривает стадо и переписывает животных в новый список rest. Если очередное животное это курица и съеденных кур пока меньше, чем кицунэ, то её имя записывается в список eaten, а в rest она не попадает, то есть курица пропадает с фермы. Остальные животные переходят в rest без изменений. Так каждая кицунэ съедает ровно одну курицу в день, причём самую старую из оставшихся, а если кур меньше, чем кицунэ, съедаются только те, что есть. Если на ферме нет кур или нет кицунэ, ничего не происходит.

```
for (Animal& animal : rest)
{
    if (toFeed > 0 && animal.getSpecies() == KITSUNE_SPECIES)
    {
        animal.feed();
        toFeed--;
    }
}
herd = rest;
```

Съеденная курица служит кормом: кицунэ, которая поела, становится менее голодной (вызывается feed()), поэтому тофу ей нужен только тогда, когда кур не осталось. Метод вызывается в конце каждого дня перед подсчётом дохода, поэтому съеденная курица в этот день дохода уже не приносит, а сообщение о том, кого съели, выводится вместе с итогами дня.

Чтобы игра выглядела веселее, в неё добавлены рисунки из обычных символов (ASCII-арт). При запуске программы показывается ферма с амбаром, силосом, деревьями, забором, облаками и солнцем. При старте игры рядом рисуются корова и курица, а когда игрок кормит корову, курицу или кицунэ либо заботится о них, показывается рисунок этого животного. Каждый рисунок хранится в своей функции (farmArt(), cowArt(), chickenArt() и kitsuneArt()) в виде многострочной строки, в которой не нужно экранировать обратные слэши. Нужный рисунок по названию вида выбирает функция animalArt():

```
string animalArt(const string& species)
{
    if (species == "Корова")
        return cowArt();
    if (species == "Курица")
        return chickenArt();
    if (species == KITSUNE_SPECIES)
        return kitsuneArt();
    return "";
}
```

Для видов без рисунка (свинья, овца и новые виды из настроек) возвращается пустая строка, поэтому игра работает с любыми животными. Функция sideBySide() ставит два рисунка рядом и выравнивает их по нижнему краю, так при старте игры рядом оказываются корова и курица.

Тестирование

Тесты, как и раньше, написаны на C++ в файле tests/test_main.cpp без сторонних библиотек. Теперь это настоящие unit-тесты классов: каждый метод Food, Animal и Player проверяется отдельно, без меню и ввода. Например, тест создаёт животное, кормит его и проверяет, что голод стал равен нулю, а игрока без корма пробует заставить кормить животное и проверяет, что действие не потратилось.

Кроме того, проверяется игра целиком: тест подаёт на вход последовательность номеров пунктов, как будто их вводит пользователь, и ищет нужные фразы в выводе. Так проверены покупка корма и животных, кормление, забота, нехватка денег, неверный ввод, гибель животных от голода и полное прохождение всех семи дней. Отдельно проверено, что настройки влияют на игру: от сложности зависит число действий и стартовые деньги, а стадо создаётся из животных, указанных в файле настроек. Тесты из второй лабораторной работы (создание и чтение файла, испорченный файл, добавление нового вида) сохранены.

Всего выполнена 161 автоматическая проверка, все проходят. Среди них проверки рисунков (они должны состоять только из обычных символов и помещаться в окно) и проверки кицунэ: она съедает по одной курице в день, съеденная курица пропадает из стада,

а без кицунэ или без кур никого не съедают. Чтобы убедиться, что тесты что-то проверяют, в код специально вносились ошибки (корм не списывался, не начислялся доход, голод не рос, сложность не влияла на правила, кицунэ никого не съедала), и в каждом случае тесты выдавали FAIL.

Доработка по замечаниям преподавателя
В версии 3.1 сохранён механизм kitsuneEats(): в конце каждого дня каждая кицунэ съедает не более одной курицы. Курицу исключают из стада; в отчёте выводится её имя. Если куриц меньше, чем кицунэ, съедается только имеющееся количество. Если кур нет, стадо не меняется.
Для индивидуальных имён добавлен метод Animal::setName() и метод Player::renameAnimal(index, proposedName). В игровом меню появился пункт 6 «Дать имя / переименовать животное». Игрок выбирает животное и вводит новое имя. Пробелы по краям удаляются; пустые, повторяющиеся и слишком длинные имена отклоняются. Переименование бесплатно и не уменьшает число действий за день. Имена используются в списке животных, сообщениях о кормлении и в отчёте кицунэ.
Тестирование доработки: сохранены исходные проверки, добавлены проверки успешного переименования, пустого и повторного имени, ошибочного индекса, ограничения длины, сохранения вида, отсутствия расхода действий, отображения имени в меню и кормлении, а также имени курицы, съеденной кицунэ. Результат запуска: 161 проверка пройдена, 0 провалено.
Дополнительные теоретические вопросы к лабораторной №3
1. Обёрточные классы. Класс-обёртка инкапсулирует значение или ресурс и предоставляет контролируемый интерфейс работы с ним. В Java стандартные классы-обёртки Integer, Double, Boolean и Character представляют примитивные значения объектами. В C++ прямого полного аналога системы Java wrapper classes нет: например, std::string оборачивает строковые данные и управляет памятью, std::unique_ptr — владением ресурсом. Не следует путать классы-обёртки Java с контейнерами std::vector и std::map.
2. Класс std::string и его методы. std::string — строковый класс стандартной библиотеки C++ из <string>. size() и length() возвращают длину (в байтах/элементах char), empty() проверяет пустоту, substr(pos, count) извлекает подстроку, find(text) ищет фрагмент, compare() сравнивает строки, append() и += добавляют текст, insert() вставляет, erase() удаляет, replace() заменяет, c_str() возвращает C-строку. В программе используются find(), substr(), size(), empty(), compare() и getline(). В UTF-8 русская буква обычно занимает несколько байт, поэтому size() не равно числу отображаемых символов.
3. Конструкторы и их виды. Конструктор вызывается при создании объекта и инициализирует его поля. Конструктор по умолчанию может вызываться без аргументов; параметризованный принимает аргументы; копирующий создаёт объект по другому объекту того же класса; перемещающий (C++11) переносит ресурсы из временного/перемещаемого объекта; делегирующий вызывает другой конструктор этого же класса. В классах Food, Animal и Player использованы параметризованные конструкторы со списками инициализации; специальные конструкторы копирования/перемещения явно не определены.
4. Отладка программ. Отладка — систематический поиск причин ошибок, их исправление и повторная проверка. Применяют компилятор с предупреждениями (-Wall -Wextra), отладочную сборку (-g), точки останова, пошаговое исполнение, просмотр переменных, журналы, тесты и санитайзеры. Ошибки делят на ошибки компиляции, выполнения и логики. В лабораторной функциональность проверяется автоматическими тестами, включая граничные случаи: пустое имя, неправильный индекс и отсутствие куриц.
5. Экстремальное программирование (XP). Гибкая методология разработки с короткими итерациями, постоянной обратной связью, простым дизайном и поддержанием высокого качества кода. Характерные практики: парное программирование, разработка через

тестирование (TDD), рефакторинг, непрерывная интеграция, частые небольшие релизы и коллективное владение кодом. В данной работе применимы небольшие изменения функциональности и автоматическая регрессия: после добавления имён повторно запускается весь набор тестов.

Контрольные вопросы

Алгоритм работы написанной программы

При запуске загружаются настройки из файла settings.txt (или создаётся файл по умолчанию). Дальше работает главное меню. При выборе «Начать игру» по уровню сложности берутся правила, из настроек создаётся стадо животных и стартовые запасы корма. Затем идут дни: в каждом игрок выбирает действия, пока они не кончатся, а в конце дня кицунэ (если она есть на ферме) съедает курицу, начисляется доход, растёт голод и погибают самые голодные животные. После последнего дня или гибели всех животных выводятся итоги, и программа возвращается в главное меню.

Классы, объекты, поля, методы

Класс это описание нового типа данных: он объединяет данные и функции для работы с ними. Объект это конкретный экземпляр класса, созданный по этому описанию. Поля (данные-члены) хранят состояние объекта, а методы это функции класса, которые работают с этим состоянием. В программе три класса: Food, Animal и Player. Например, Animal это класс, а «Корова 1» и «Курица 3» объекты этого класса. Поля Animal это вид, имя, голод и настроение, а методы feed(), care() и passDay().

Основные принципы ООП

Основных принципов четыре. Инкапсуляция скрывает данные внутри класса и открывает доступ к ним только через методы. Наследование позволяет создавать новый класс на основе существующего и переиспользовать его поля и методы. Полиморфизм позволяет работать с объектами разных классов через единый интерфейс, например вызывать одну и ту же функцию у разных потомков. Абстракция выделяет в объекте только существенные для задачи свойства. В моей работе использована инкапсуляция: поля классов закрыты в private, а изменить их можно только методами. Наследование и полиморфизм в программе не применялись, для этой задачи хватило трёх независимых классов.

Конструкторы, их виды, различия

Конструктор это специальный метод, который вызывается при создании объекта и задаёт его начальное состояние. Конструктор по умолчанию не принимает параметров. Параметризованный конструктор принимает значения для инициализации полей: в программе такие у всех трёх классов, например Animal(вид, имя). Конструктор копирования создаёт новый объект как копию другого, он вызывается, например, когда объект передаётся по значению или кладётся в vector, в моей работе он создаётся

компилятором автоматически. Есть также конструктор перемещения (C++11), который забирает данные у временного объекта вместо копирования, и делегирующий конструктор, который вызывает другой конструктор того же класса. Различаются они тем, откуда берутся данные для нового объекта.

Отладка программ

Отладка это поиск и исправление ошибок в программе. Ошибки бывают синтаксические (программа не компилируется), ошибки времени выполнения (например, выход за границы массива) и логические (программа работает, но результат неверный). Для поиска ошибок используют отладчик: он позволяет ставить точки останова, выполнять программу по шагам и смотреть значения переменных. Также помогает вывод промежуточных значений на экран и автоматические тесты. В моей работе ошибки искались с помощью тестов: если тест падал, по названию было видно, какая часть логики сломана.

Экстремальное программирование

Экстремальное программирование (XP) это гибкая методология разработки. Её основные практики: короткие итерации и частые выпуски, написание тестов вместе с кодом (иногда до него), простой дизайн, постоянная переделка кода для улучшения его структуры (рефакторинг), парное программирование, непрерывная интеграция и тесное общение с заказчиком. В этой лабораторной работе близка к XP практика автоматического тестирования: тесты запускаются на GitHub при каждом Pull Request, а программа развивается небольшими шагами от лабораторной к лабораторной.

Вывод

В ходе работы в программу добавлена основная логика: пункт «Начать игру» запускает симулятор ухода за животными. Реализованы классы Food, Animal и Player, разные виды животных едят разный корм, а игра использует настройки из предыдущих лабораторных работ: уровень сложности и список животных из файла settings.txt. Я закрепил работу с классами, конструкторами и инкапсуляцией. Дополнительно в игру добавлены ASCII-рисунки и животное кицунэ, которое съедает по одной курице в день. Для проверки выполнена 161 автоматическая проверка, все проходят.